

A Survey of Python Type Analysis: Techniques, Challenges, and Emerging Directions

1st Mingyeong Jeong

Computer Science and Engineering
Chungnam National University
Daejeon, South Korea
jmink002@o.cnu.ac.kr

2nd Sungho Lee

Computer Science and Engineering
Chungnam National University
Daejeon, South Korea
eshaj@cnu.ac.kr

Abstract—The dynamic nature of Python poses fundamental challenges for accurate type inference, driving extensive research on type inference, checking, and recovery across static, dynamic, learning-based, and hybrid paradigms. This survey provides a comprehensive analysis of major Python type analysis efforts spanning static systems, dynamic monitoring frameworks, and emerging machine-learning-based models. We systematically identify and analyze representative studies collected through a keyword-driven search within the programming languages and software engineering domains, selecting those that explicitly address Python or its dynamic typing characteristics. This survey categorizes the Python type analysis landscape into four approaches: static analysis, dynamic monitoring, hybrid methods that combine static and runtime information, and AI-driven inference based on large language models. It provides a detailed review of the technical principles, advantages, limitations, and open challenges of each approach, with particular attention to Python-specific issues such as duck typing, flow-sensitive type changes, reflection, dynamic code generation, and heterogeneous containers. Our analysis shows that no single approach is sufficient to fully address Python type reasoning; instead, combining static, dynamic, and AI-based techniques in complementary ways is emerging as a promising direction for future research. By synthesizing existing research and tools, this survey clarifies the current landscape of Python type analysis and provides guidance for both practitioners and researchers seeking robust, practical, and forward-looking solutions.

Index Terms—Python, Type analysis, Static typing, Dynamic typing, Hybrid, Machine learning

I. INTRODUCTION

Python has experienced significant growth over the past decade and is now one of the most widely used programming languages worldwide. Its popularity has sharply increased since 2018 and it has consistently ranked as the most popular language in multiple surveys since 2022 [1]. This rise stems from Python’s versatility across domains such as web development, data analytics, scientific computing, automation, and especially artificial intelligence and machine learning. The expansion of AI ecosystems—supported by deep learning frameworks (e.g., TensorFlow [2], PyTorch [3]) and natural language processing libraries (e.g., Hugging Face Transformers [4])—has further accelerated its adoption among both industry and research communities.

In addition to its broad applicability, Python’s concise and natural-language-like syntax lowers the barrier to entry for

new developers. Features such as dynamic typing, high-level data structures, and rich standard libraries allow programmers to express complex ideas with minimal boilerplate code and it allows rapid prototyping. As a result, Python has become a first-choice language for prototyping, experimentation, and educational purposes, fostering a vast ecosystem of open-source libraries and active developer communities.

However, Python’s dynamic and interpreted nature introduces significant challenges for program reliability. Unlike compiled languages, where many errors can be caught at compile time, Python programs often surface errors only during execution. Among these runtime errors, type-related issues such as `TypeError` and `AttributeError` are particularly prevalent, accounting for a substantial portion of all reported bugs—studies suggest approximately 30% of runtime errors in Python stem from type mismatches [5]. This prevalence of type errors not only affects software reliability but also complicates maintenance and refactoring of large codebases, where inconsistent or unexpected type usage can propagate subtle bugs across modules [6].

Consequently, type checking and type analysis have become critical for improving the reliability and maintainability of Python programs. The introduction of optional type annotations in Python 3.5 provided a mechanism for developers to express expected types explicitly, enabling static verification while preserving the flexibility [7]. Tools such as `mypy` [8], `Pyre` [9], and `Pyright` [10] leverage these annotations to detect type inconsistencies before execution, reducing the likelihood of runtime failures. Beyond static checking, hybrid approaches combining static and runtime monitoring, as well as emerging AI-based techniques, have begun to address Python’s dynamic and flexible behavior more effectively.

Despite these advancements, Python type analysis remains challenging due to several unique characteristics of the language. First, Python follows a form of structural typing, often referred to as “duck typing,” where object compatibility is determined by available attributes and methods rather than nominal type hierarchies. Second, variable types in Python can change dynamically depending on the control flow, requiring type inference mechanisms to be flow-sensitive to achieve accuracy. Third, Python supports advanced dynamic features, including reflection, runtime code generation through `eval`

and `exec`, and dynamic module loading, all of which complicate static reasoning. Finally, Python data structures such as lists, dictionaries, and sets frequently contain heterogeneous elements whose types may vary over time, making traditional uniform-type assumptions insufficient. These language characteristics collectively create a significant gap between static type assumptions and the actual behavior of programs, motivating the development of specialized analysis techniques.

In this context, the present study aims to provide a comprehensive survey of Python type analysis techniques and tools, categorizing them according to their underlying methodologies and examining their strengths and limitations. Specifically, this paper addresses the following research questions (RQs):

- **RQ1** : How can Python’s dynamic nature be preserved while ensuring program stability through type checking?
- **RQ2** : What are the main challenges in type inference and type analysis for Python programs?
- **RQ3** : What technical characteristics and limitations define existing Python type analysis tools?

To answer these questions, we analyze Python type analysis tools spanning several paradigms, including static analyzers leveraging type annotations, runtime and hybrid monitoring systems, and recently developed AI-powered methods enabled by large language models. By systematically examining these tools and approaches, we aim to clarify the current landscape of Python type analysis, highlight open challenges, and identify promising directions for future research.

In summary, this paper contributes a structured overview of Python type analysis, emphasizing the interplay between dynamic flexibility and the need for program reliability, and provides insights for both tool developers and researchers aiming to advance Python’s type-checking ecosystem.

II. BACKGROUND

A. Python Type System

Python is a dynamically typed language in which variable types are determined at runtime rather than at the point of declaration. Because a variable’s type is defined by the value assigned during execution, its type can change at any point while the program runs. Moreover, Python adopts a pure object-oriented design in which “everything is an object,” making the notion of object compatibility a central issue in type analysis. In dynamic languages such as Python, object compatibility is governed by duck typing. Under duck typing, compatibility is determined not by an object’s nominal type (e.g., `int`, `str`, `Parrot`) but by whether it provides the required methods or attributes (such as `fly()` or `len()`), regardless of the class hierarchy or inheritance relation.

Listing 1 illustrates this principle. As shown in lines 21, 24, instances of `Parrot` and `Airplane`—despite having no inheritance or structural relationship—can be passed to the same function `lift_off()` without causing any errors. This demonstrates that compatibility is determined by the presence of the `fly()` method rather than by the concrete classes `Parrot` or `Airplane`. In contrast, when a `Whale` object, which does

Listing 1 Example Python program demonstrating object compatibility determined by duck typing.

```

1  class Parrot:
2      def fly(self):
3          print("Parrot flying")
4
5  class Airplane:
6      def fly(self):
7          print("Airplane flying")
8
9  class Whale:
10     def swim(self):
11         print("Whale swimming")
12
13 def lift_off(entity):
14     entity.fly()
15
16 parrot = Parrot()
17 airplane = Airplane()
18 whale = Whale()
19
20 # prints `Parrot flying`
21 lift_off(parrot)
22
23 # prints `Airplane flying`
24 lift_off(airplane)
25
26 # Error : `Whale` object has no attribute 'fly'
27 lift_off(whale)

```

not define `fly()`, is passed to the same function (line 27), a `TypeError` is raised due to the missing attribute. This shows that Python’s type system is based on behavior—what an object can do matters more than its formal type.

Consequently, type determination in Python cannot rely solely on nominal types; it must also incorporate structural types defined by the methods and attributes available on an object. This structural dimension becomes even more complex because Python allows these methods and attributes to be dynamically added, removed, or modified during execution. For example, objects can obtain new attributes through monkey patching, classes can be modified at runtime, and dynamic features such as `__getattr__`, `__getattribute__`, decorators, and metaclasses can alter the attribute lookup process itself. As a result, the effective “shape” of an object is not statically fixed but may evolve throughout the program’s lifetime.

Such flexibility offers expressive power but poses inherent challenges for precise static type analysis. Since the availability of attributes and methods may depend on control-flow conditions, runtime configuration, external library behavior, or execution order, a static analyzer must either over-approximate the possible behaviors—leading to imprecise or overly conservative results—or risk missing valid error cases that manifest only under specific runtime conditions. Additionally, heterogeneous containers, dynamic code generation (e.g., `eval`, `exec`), and late-binding semantics further complicate the inference process. These factors collectively make Python’s type system difficult to analyze purely statically, motivating the development of hybrid and dynamic approaches that incorporate runtime observations to complement static reasoning.

Listing 2 Example Python program demonstrating object compatibility determined by duck typing.

```
1 def foo(a: int, b: int):  
2     return a + b  
3  
4 # No Error  
5 a = foo(1, 2)  
6  
7 # TypeError only in static type checker  
8 b = foo(2, 3.14)  
9  
10 # TypeError in both static type checker and runtime  
11 c = foo(3, "hello")
```

B. Evolution of Python Type System

The ability to omit explicit types in Python allows developers to write concise code and rapidly prototype programs. However, this high degree of flexibility can reduce code readability, complicate maintenance, and increase the likelihood of unintended type-related bugs. To mitigate these issues, Type Annotations were introduced in Python 3.5 for variables and in Python 3.6 for function parameters and return. Type Annotations are optional and do not affect the runtime behavior of a program, but they enable developers to explicitly state type information. This improves readability and developer productivity while serving as a form of contract that helps enhance program reliability.

Following the introduction of Type Annotations, various static type checkers have been actively developed in the Python ecosystem. Before this change, static analysis tools had to rely entirely on type inference, which is notoriously difficult in dynamic languages and often leads to imprecise analyses. With explicit annotations, static analyzers can perform significantly more accurate checks by leveraging developer-provided type information rather than depending solely on inference.

Python thus adopts a Gradual Typing paradigm [11] that combines the advantages of both dynamic and static type systems. Under Gradual Typing, parts of the program where types can be determined statically are checked statically, while the rest continue to operate under the dynamic type system. This hybrid approach allows Python to achieve both flexibility and reliability in large-scale software development. The concept of Gradual Typing illustrates with the Listing 2

In this example, the function `foo` explicitly specifies that both `a` and `b` should be of type `int`. This example highlights the following three key observations:

- `foo(1, 2)` satisfies the type specification, so neither the static type checker nor the runtime reports any errors.
- `foo(2, 3.14)` causes a type error in the static type checker because a float is passed where an `int` is expected. However, the Python runtime accepts the call and executes it, demonstrating that annotations are not enforced at runtime.
- `foo(3, "hello")` produces a type error in the static type checker, and additionally, it raises a runtime `TypeError` because Python cannot add an integer and a string.

This example illustrates how Python’s Gradual Typing performs static checking where type information is available, while maintaining dynamic typing behavior where static analysis is insufficient. This hybrid model provides the reliability of a static type system without sacrificing the flexibility of dynamic typing, making it suitable for large-scale software development.

III. RESEARCH RESULTS

In this paper, we aim to provide a comprehensive understanding of how Python type analysis has evolved and what technical strategies underpin current approaches. To this end, we systematically classify the major type analysis techniques used in Python, as illustrated in Figure 1, and compare their technical characteristics, strengths, weaknesses, and representative tools. Through this structured examination, we clarify the design trade-offs embedded in each approach, identify the fundamental limitations shared across existing techniques, and outline promising future research directions to address these challenges.

A. Select Papers

In this survey, we aim to provide a comprehensive review of research on Python type analysis and type checking. To identify relevant studies, we conducted a systematic search of the literature using keywords such as “type analysis,” “static type checking,” and “dynamic type checking.” From the retrieved studies, we selected only those that focus on Python or explicitly address the dynamic characteristics of Python programs in their type inference or type checking techniques.

Based on these criteria, the survey includes traditional static type analyses [12], constraint-based approaches [13], machine learning–driven methods [14]–[16], and hybrid techniques combining static analysis and learning [17]–[19]. In addition, we also consider studies that aim to predict or repair Python runtime type errors [5], [20]–[22], covering research that extends beyond pure type inference to practical program improvement.

The selected studies collectively address diverse aspects of Python type analysis, including handling dynamic features, compound data types, and flow sensitivity. They represent a broad spectrum of methodologies—static, dynamic, and learning-based approaches—and are thus well-suited for a comprehensive comparison and discussion of the current state and future directions of Python type checking research.

B. RQ1 : How can Python’s dynamic nature be preserved while ensuring program stability through type checking?

Because Python is a dynamically typed language, the actual structure and state of objects are determined only at runtime. While this design provides significant flexibility and expressiveness for developers, it inevitably reduces the precision attainable through static analysis and increases the likelihood of runtime errors such as `TypeError` and `AttributeError`. As a result, the research community has proposed a wide range

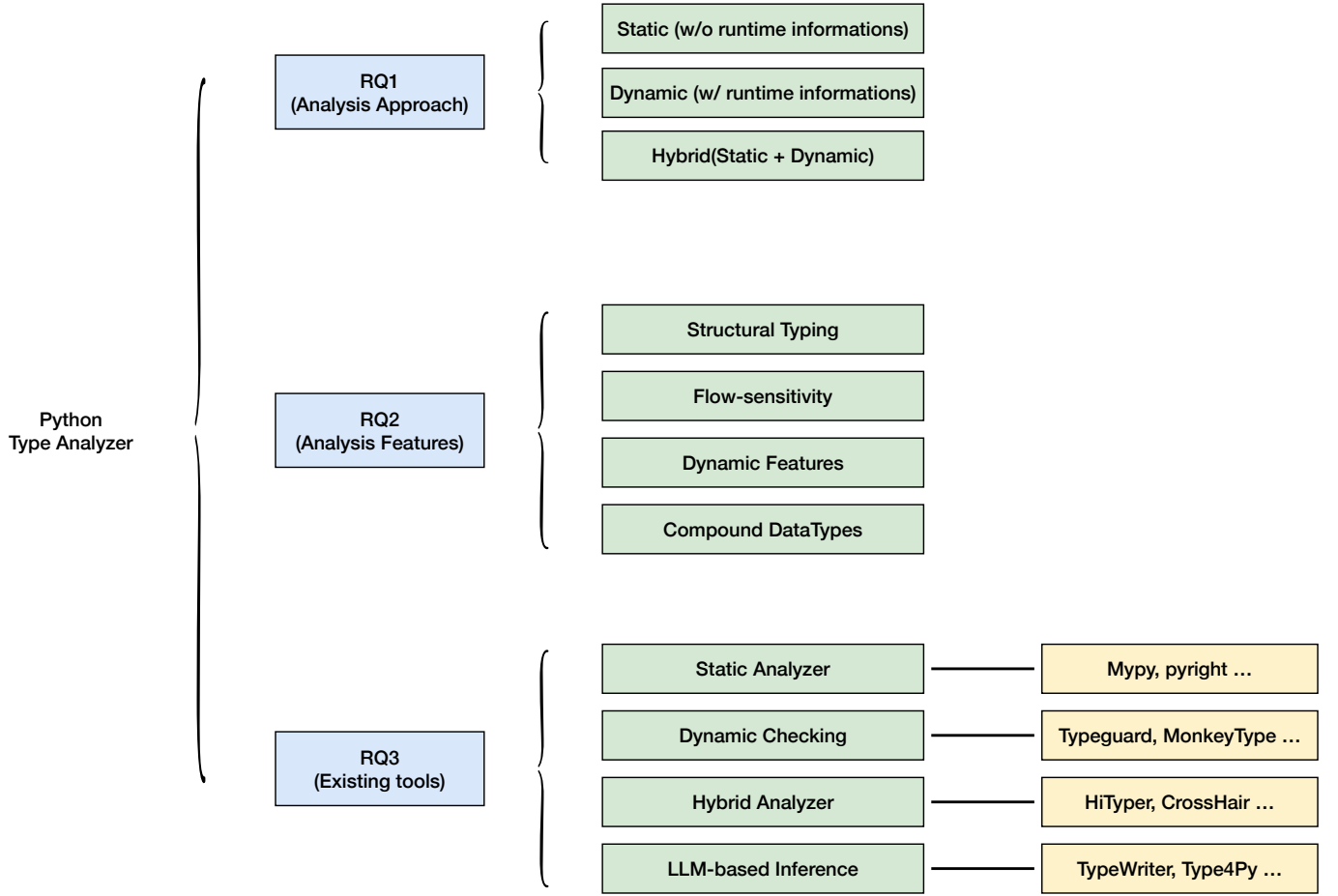


Fig. 1: Overview of Python type analysis in relation to the research questions (RQ1–RQ3).

of analysis techniques that attempt to improve program reliability while still preserving Python’s dynamic nature. These approaches vary widely in their assumptions, goals, and underlying technical mechanisms—from purely static inference to hybrid and runtime-assisted analyses—reflecting the difficulty of balancing soundness, precision, and usability in such a dynamic environment.

Table I summarizes the number of existing type checking approaches, providing an overview of the landscape. The following sections examine each category in detail and analyze their strengthes and limitations.

TABLE I: Number of papers and tools classified by Type Checking Approaches (RQ1).

Static	Dynamic	Hybrid
21	2	4

1) Static Type Checking with Type Annotations: Since the introduction of type annotations, it has become possible to enhance program reliability using explicit static type information. Static type checkers such as Mypy [8] and Pyre [9] were developed on this basis, enabling the detection of potential

`TypeError` in variables, function parameters, and return values before execution. Moreover, Python adopts a gradual typing approach, allowing static verification even when only parts of a program are annotated. This enables developers to strengthen type safety incrementally. Typically, static type checking verifies, as in Algorithm 1, whether the code matches the provided type annotations and may further perform type inference to ensure correct use of types in practice.

Table I shows that static approaches constitute the largest portion of existing research and tools. This prevalence can be attributed to two factors. First, static type checking imposes no runtime overhead, making it attractive for industrial use where performance is critical. Second, type annotation support in Python has steadily matured—particularly with the introduction of type hints (PEP 484), generics (PEP 585), and gradual typing infrastructures—providing a well-defined foundation upon which many tools have been developed. As a result, static analysis has become the most accessible and practically deployable method for improving Python’s reliability.

However, because static analysis is limited to annotated regions, it cannot provide the same level of safety guarantees as fully statically typed languages. Furthermore, Python’s type annotations rely primarily on nominal typing, which limits

Algorithm 1 GradualTypeCheck

Require: Python function f with optional type annotations, arguments $args$

Ensure: TypeError if type mismatch detected

```
1: for each argument  $a_i$  in  $args$  do
2:   if  $f$  has type annotation for  $a_i$  then
3:     if  $\text{type}(a_i) \neq \text{annotation\_type}(a_i)$  then
4:       raise TypeError
5:     end if
6:   end if
7: end for
8:  $\text{result} = \text{execute}(f, args)$ 
9: if  $f$  has return type annotation then
10:  if  $\text{type}(\text{result}) \neq \text{annotation\_type}(f.\text{return})$  then
11:    raise TypeError
12:  end if
13: end if
14: return result
```

their ability to accurately capture Python’s structural and behavior-based typing semantics, such as duck typing based on the presence of specific methods or attributes. Therefore, while static type checking has the advantage of detecting errors in advance without incurring runtime overhead, it suffers from a high rate of false positives due to its difficulty in capturing Python’s dynamic behavior.

2) *Dynamic Monitoring and Runtime Enforcement:* Dynamic type monitoring and runtime enforcement observe type-related constraints during the actual execution of a program and report violations immediately. Although this approach cannot capture all errors prior to execution—as static analysis attempts to do—it provides high accuracy for type violations along concrete execution paths. In languages such as Python, where dynamic binding is pervasive, this approach is particularly practical because it can directly reproduce behavior-based runtime errors that static techniques often fail to detect.

The most widely used runtime enforcement mechanism is decorator-based wrapping. By attaching a type-checking decorator to a function, as in Algorithm 2, the checker verifies at each call whether the argument and return values conform to the declared type annotations. Libraries such as `typeguard` [23] implement this strategy using `typing.get_type_hints()` and `isinstance`-based checks. This method requires minimal modification to existing code and detects mis-typed values precisely at the point of failure, making it highly effective for debugging. However, because checks are executed on every function call, runtime overhead is unavoidable, and complex type structures—such as generics, nested container types—cannot always be validated at runtime.

Another form of dynamic monitoring leverages runtime traces for downstream analyses. `MonkeyType` [24], for example, records observed argument and return types during test execution and automatically generates `Mypy`-compatible annotations. Because this approach captures real execution behavior, it is particularly useful for inferring types in legacy

systems or analyzing complex functions. Nevertheless, it critically depends on test quality and coverage; types for rarely executed paths or untested branches remain unknown.

Dynamic enforcement therefore provides high precision on observed paths but suffers from fundamental limitations. Achieving comprehensive program safety requires sufficiently broad test or execution coverage, creating substantial coverage dependency and potentially high execution cost. Furthermore, runtime information alone is often insufficient to accurately validate deeply nested or generic types, protocol-based structural constraints, or types involving implicit invariants. Techniques such as conditional checks, sampling-based monitoring, or partial instrumentation can mitigate overhead, but they cannot fully eliminate these inherent weaknesses.

In practice, Python’s dynamic analysis ecosystem reflects these trade-offs. `typeguard` offers decorator-based runtime checking, `MonkeyType` synthesizes annotations from runtime traces, and `CrossHair` [25] combines dynamic execution with symbolic reasoning for hybrid verification. Collectively, these tools complement static checkers by addressing classes of errors that remain challenging to detect in a highly dynamic language such as Python.

Algorithm 2 RuntimeTypeMonitor

Require: Python function f with optional type annotations

Ensure: Detect type violations during execution

```
1: Wrap  $f$  with a type-checking decorator
2: On each call to  $f$ :
3:  for each argument  $a_i$  do
4:    if type annotation exists for  $a_i$  then
5:      if  $\text{type}(a_i) \neq \text{annotation\_type}(a_i)$  then
6:        Log type violation
7:      end if
8:    end if
9:  end for
10: Execute original function  $f$ 
11: if return type annotation exists then
12:  if  $\text{type}(\text{return}) \neq \text{annotation\_type}(\text{return})$  then
13:    Log type violation
14:  end if
15: end if
```

3) *Hybrid Approaches:* To address both Python’s dynamic nature and the inherent limitations of static analysis, recent research has increasingly focused on hybrid analysis, which integrates static and dynamic techniques into a unified framework. By combining the global reasoning power of static analysis with the execution-based precision of dynamic monitoring, hybrid methods aim to provide more practical and robust guarantees for Python programs.

A familiar instance of such integration is the combination of gradual typing with runtime checking. Tools such as `Mypy` and `Pyre` statically verify annotated code regions while validating unannotated or ambiguous segments at runtime, thereby reducing static false positives and improving overall reliability. Nevertheless, these approaches are not counted as hybrid in

Table I, because they rely primarily on annotation-driven static checks combined with simple runtime guards, rather than incorporating substantial runtime information into the analysis process.

In Table I, hybrid approaches refer specifically to techniques that use both compile-time and runtime information as essential components of the analysis. Examples include methods that enrich static constraints using execution traces, that integrate dynamic attribute access patterns into static reasoning, or that iteratively refine abstractions using runtime-observed types. Likewise, analyses that combine symbolic reasoning with concrete execution fall within this category. Conversely, AI-based techniques trained solely on static code features are excluded because they utilize only compile-time information.

By leveraging dynamic observations to refine or correct the conservatism of static analysis, hybrid methods can reflect Python’s real-world usage patterns more faithfully and significantly reduce spurious warnings. At the same time, static analysis provides a global viewpoint that compensates for the path-specific nature of dynamic monitoring, enabling broader safety guarantees that cannot be achieved by runtime techniques alone. This combination often yields higher type safety, improved precision, and greater tool usability than either approach by itself.

However, hybrid methods also inherit several challenges. Because they depend on runtime information, their effectiveness is tied to the availability and coverage of executed paths; insufficient execution data can limit their precision. Runtime monitoring and trace collection introduce performance overhead, and the integration of static and dynamic information imposes additional computational and engineering costs. Despite these limitations, hybrid analysis remains one of the most promising research directions for Python type analysis, offering a balanced and practical means of handling the language’s highly dynamic semantics.

4) Summary: Landscape of Type Analysis Techniques for Python: Because Python is fundamentally a dynamically typed language, fully static type checking—of the kind possible in statically typed languages—is structurally infeasible. Object attributes may be added, removed, or modified at runtime, and even the return type of a single function may vary depending on control flow. Despite these challenges, recent advances in type analysis have demonstrated that Python can achieve a meaningful degree of type safety through a combination of complementary techniques.

As discussed in the previous sections, existing approaches fall into three major directions, each with distinct strengths and limitations. First, annotation-based static analysis performs ahead-of-time verification and can detect errors early without incurring runtime costs. Its global reasoning capability allows broad safety guarantees. However, its inherent conservatism and difficulty in modeling Python’s dynamic behavior often lead to false positives, particularly in unannotated or highly dynamic regions. Second, runtime monitoring and enforcement provide high precision for observed execution paths and

faithfully capture dynamic semantics that static methods miss. Yet these techniques depend heavily on execution coverage, introduce runtime overhead, and cannot ensure global safety unless extensive tests or traces are available. Third, hybrid approaches integrate static and dynamic information, allowing static analysis to be refined using concrete runtime observations. This reduces false positives, increases practical precision, and better reflects real-world usage patterns. At the same time, hybrid analysis still inherits certain limitations—such as coverage dependence and the cost of merging diverse information sources—but offers a balanced compromise between static safety and dynamic flexibility.

In summary, while no single technique can provide complete type safety for Python, these three categories collectively define the current landscape of Python type analysis. Among them, hybrid approaches are gaining prominence, as they leverage the complementary strengths of static reasoning and dynamic evidence to address Python’s dynamic nature more effectively than either method alone.

Answer to RQ1

Recent work shows a clear shift from purely static analysis toward hybrid and runtime-assisted approaches for Python type checking. Static analyzers still play a central role in large-scale verification, but runtime monitoring complements them by capturing behaviors that static methods miss. As a result, hybrid systems have become the most practical way to balance Python’s dynamic nature with meaningful type safety.

C. RQ2 : What are the main challenges in type inference and type analysis for Python programs?

Type analysis in Python is fundamentally more challenging than in traditional statically typed languages due to its structural and dynamic nature. In particular, (1) structural typing-based object compatibility, (2) flow-sensitive types, (3) dynamic features such as reflection and dynamic code generation, and (4) heterogeneous containers are frequently cited as the main obstacles. Table II summarizes whether existing research and tools support these four Python-specific challenges. This provides a clear overview of the strengths and limitations of current Python type analysis approaches.

First, Python follows a duck-typing model that is closer to structural typing than nominal typing. The meaning of an object is determined by the methods and attributes it provides, making it difficult to accurately model objects using only static type names. Most existing studies do not use structural typing; instead, they perform analysis based solely on type names (nominal typing). For this reason, current approaches cannot dynamically model Python’s types, leading to high uncertainty in static type inference.

Second, Python variables may take different types depending on control flow. A single variable may assume different types across different branches, making it difficult for traditional static analysis to infer a consistent type. To address

TABLE II: RQ2: Table showing whether existing Python type analysis papers and tools support key Python features, including structural typing, flow-sensitive types, dynamic features, and compound data types. (✓ is support, ✗ is not.)

Paper/Tool	Structural Typing	Flow-sensitivity	Dynamic Features	Compound DataTypes
[12]	✗	✗	✗	✓
[26]	✗	✗	✗	✓
[13]	✗	✓	✗	✓
[27]	✗	✗	✓	✓
[28]	✗	✗	✓	✓
[29]	✗	✗	✓	✓
[15]	✗	✗	✓	✓
[20]	✗	✓	✓	✓
[30]	✗	✓	✓	✓
[17]	✗	✓	✓	✓
[31]	✗	✗	✓	✓
[14]	✗	✗	✓	✓
[18]	✗	✗	✓	✓
[21]	✗	✓	✓	✓
[19]	✗	✓	✓	✓
[32]	✗	✓	✗	✓
[16]	✗	✗	✓	✓
[5]	✓	✓	✓	✓
[22]	✓	✓	✓	✓
[24]	✗	✗	✓	✓
[25]	✗	✗	✓	✓
[23]	✗	✗	✓	✓
[33]	✓	✓	✓	✓
[8]	✗	✗	✗	✓
[9]	✗	✓	✓	✓
[34]	✗	✓	✓	✓
[10]	✗	✓	✓	✓
27	3	13	22	27

this, recent studies perform flow-sensitive analysis, but only about 50% of the works apply it; the rest rely on methods derived from static-typed languages that do not account for control flow. Flow-insensitive analysis sometimes uses techniques such as Static Single Assignment (SSA) to represent control flow. However, increasing precision using these methods rapidly raises computational costs. Therefore, flow-aware analysis is essential, and balancing accuracy with performance remains a central challenge in Python type analysis.

Third, Python’s powerful dynamic features—reflection and dynamic code generation—fundamentally threaten the soundness of any static type analysis. Dynamic attribute access via `__getattr__` and `__getattribute__`, runtime code generation using `eval()` and `exec()`, dynamic module loading through `importlib`, and metaclass-based class generation all defer program structure determination until runtime. Accurately analyzing these behaviors would require near-complete program execution, making precise static inference difficult within traditional frameworks. As shown in Table II, about 80% of studies are marked as supporting dynamic features; however, this criterion is intentionally broad—any support for even a subset of Python’s diverse dynamic behaviors is counted. This is because no existing work fully handles all dynamic features, and achieving comprehensive support is effectively infeasible without execution-based techniques.

Finally, Python’s lists, sets, and dictionaries inherently

allow heterogeneous elements. A single list may contain integers, strings, and user-defined objects, and element types may evolve during execution. Simple uniform type models are insufficient, and more sophisticated representations—such as union types, intersection types, or lattice-based type spaces—are required. All surveyed studies and tools account for this feature, highlighting that heterogeneous containers are a critically important aspect of Python type analysis.

Answer to RQ2

Python type analysis exhibits high complexity due to (1) structural typing-based non-uniformity, (2) types that change continually depending on control flow, (3) the possibility of runtime code generation, and (4) heterogeneous containers. These characteristics make it difficult for traditional static analysis frameworks to achieve accurate and sound modeling, highlighting the need for new analysis paradigms tailored to Python’s inherently dynamic nature.

D. RQ3 : What technical characteristics and limitations define existing Python type analysis tools?

The Python ecosystem includes a wide range of type analysis tools, spanning static, dynamic, hybrid, and, more recently, LLM-based approaches. Each category adopts a

distinct technical strategy to handle Python’s highly dynamic language characteristics, resulting in a mix of complementary strengths and inherent limitations. This section classifies major tools into four categories and analyzes their technical features and constraints.

Static type checkers such as Mypy, Pyright, and Pyre are the most widely used tools in practice. These systems construct a type environment primarily from user-provided type annotations and verify type consistency through constraint solving. They support advanced features of Python’s typing system—such as Protocols and TypedDicts—to enable structural subtyping and partial type inference. This design yields fast analysis performance and reasonable precision for large-scale codebases, making static analysis highly practical. However, their heavy reliance on annotations means that accuracy degrades sharply in real-world industrial code, where annotations are often missing or incomplete. They also cannot fully model Python’s dynamic features such as reflection, monkey patching, and dynamic attribute creation, resulting in unavoidable false positives and negatives.

In contrast, dynamic and hybrid analysis tools leverage runtime information to achieve higher accuracy. Tools such as Typeguard and contract-based libraries validate arguments, return values, and structural properties of objects at execution time, allowing detection of type violations soundly during actual runs. Tools like MonkeyType collect type hints from execution traces, enabling high-precision, test-driven type inference. These approaches require little to no annotation but structurally depend on test coverage; unexecuted paths cannot be analyzed. Runtime instrumentation also introduces performance overhead. Recent academic work explores advanced static techniques, including constraint-based analysis, symbolic execution, and SSA-based type analysis, employing refined type models such as union-type lattices, intersection types, and path-sensitive constraint solving to achieve higher precision [22]. However, these techniques are computationally expensive and difficult to apply to real-world codebases, especially in the presence of Python’s irregular dynamic features.

Finally, AI-based type analysis—using models such as GPT variants, CodeLlama, or specialized type-inference models—has emerged as a new direction. AI-based methods can infer types with high coverage even for minimally annotated codebases and can perform semantic-level reasoning that detects inconsistencies overlooked by traditional static analyzers. However, AI models operate as black boxes; their reasoning is opaque, prone to hallucination, and fundamentally unable to guarantee soundness or completeness. Moreover, AI-driven type inference remains sensitive to training data quality and lacks explicit formal guarantees.

The changing trends in Python type checking research can be observed in Figure 2. Over the years, the use of AI-based approaches has gradually increased, and notably, hybrid methodologies that combine static analysis with AI to ensure type soundness are being actively explored.

In summary, Python type analysis tools exhibit complementary strengths depending on their underlying analysis

paradigm, but no single approach can fully capture the complexity of Python’s dynamic semantics. Static analysis is efficient yet raise a lot of false positive, dynamic analysis is precise yet coverage and cost-constrained, and AI-based methods are flexible but lack soundness. Consequently, recent research trends increasingly favor hybrid approaches that combine static, dynamic, and AI-based techniques. This observation underscores that Python type analysis is not a problem solvable by any single method.

Answer to RQ3

Python type analysis tools can be classified into static, dynamic, research-oriented, and LLM-based approaches. Static analysis is fast but struggles with dynamic features, while dynamic and hybrid methods offer accuracy at the cost of coverage and runtime overhead. LLM-based approaches enable semantic-level reasoning but lack determinism and soundness. Overall, no single technique is sufficient, and hybrid combinations of static, dynamic, and AI-based methods are becoming increasingly important.

IV. DISCUSSION

This paper reviews the Python type analysis ecosystem and compares how static, dynamic, and AI-based approaches address core challenges while exposing their limitations. The analysis clarifies how Python’s dynamic features shape the design of these techniques. As shown in **RQ1**, achieving comprehensive safety with static methods alone is difficult due to Python’s dynamic object model, flexible typing, and runtime-dependent behavior. As a result, static checking, runtime monitoring, and hybrid or feedback-driven methods coexist, forming a layered landscape. This multi-paradigm ecosystem reflects a practical balance between developer productivity, Python’s dynamic flexibility, and the need for reliability and early error detection.

The structural challenges identified in **RQ2** further illustrate why no single analysis method is sufficient. Structural typing, flow-sensitive type changes, heterogeneous containers, and dynamic code generation fundamentally diverge from assumptions embedded in conventional type theory—such as nominal hierarchies, globally stable object layouts, and the closed-world assumption. These characteristics require analyzers to reexamine foundational principles rather than simply adapting techniques from statically typed languages. For instance, flow-sensitive analysis significantly improves precision, yet incurs substantial computational cost; flow-insensitive approaches scale well but fail to capture control-flow-dependent behaviors. This trade-off highlights the persistent tension among precision, scalability, and soundness that all Python analyzers must navigate. Furthermore, the practical inability to statically approximate dynamic features such as reflection, runtime attribute creation, or eval-driven code generation reinforces the need for complementary techniques that operate across both compile-time and runtime contexts.

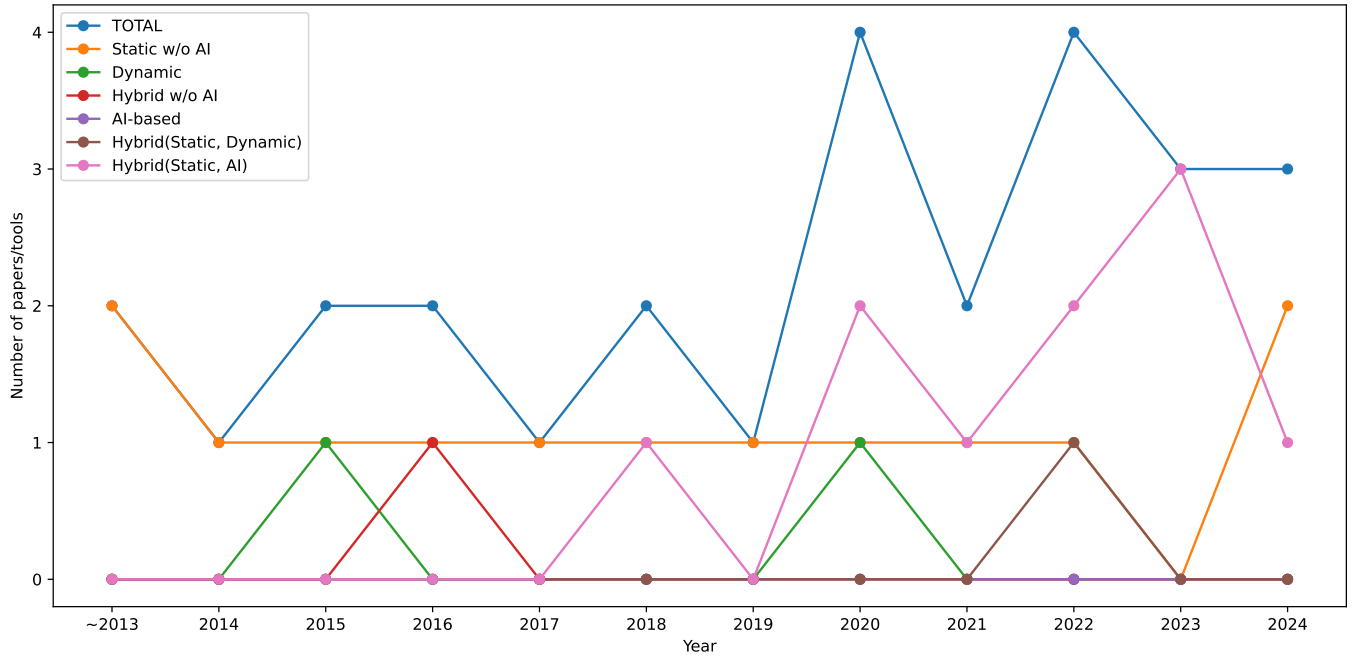


Fig. 2: Number of Python type analysis papers and tools developed per year (2013–2024) by methodology. This line chart illustrates the annual trends in the development of research and tools across different type checking approaches, including static, dynamic, hybrid, and AI-based methods.

The tool comparison in **RQ3** reflects this broader landscape. Static analyzers remain indispensable for large-scale and industrial codebases due to their speed and integration with development workflows. However, their reliance on annotations and limited capacity to represent dynamic constructs restrict their precision and completeness. Dynamic and hybrid tools mitigate these limitations by incorporating runtime information, enabling high accuracy on executed paths and capturing behaviors that static analysis cannot predict. Yet these tools necessarily depend on execution coverage and incur runtime overhead. Advanced research tools based on constraint solving, symbolic execution, or SSA-based type modeling provide sophisticated reasoning capabilities, but face challenges in industrial adoption due to computational cost and difficulty handling unstructured dynamic behaviors. Meanwhile, AI-based techniques—particularly LLM-powered reasoning—demonstrate strong performance in low-annotation settings and enable type inference even for highly dynamic idioms. Nevertheless, their opaque reasoning processes, susceptibility to hallucination, and lack of soundness guarantee limit their standalone applicability in safety-critical contexts.

Across **RQ1–RQ3**, a clear pattern emerges. Python’s dynamic, open-ended semantics create a fundamental gap between what traditional static analysis can guarantee and what developers require in practice. Each paradigm—static, dynamic, hybrid, and AI-based—offers partial strategies to bridge this gap, yet none is complete on its own. Hybrid and feedback-driven approaches increasingly appear to provide the most promising pathway forward. These methods integrate

compile-time reasoning, runtime observation, and AI-guided inference, enabling analyses that are both more precise than static techniques and more comprehensive than dynamic monitoring alone. Moreover, hybrid models can incorporate AI-derived insights while applying static constraints as guardrails, alleviating issues of soundness and hallucination.

From a broader perspective, this survey highlights several implications for future tool and research design. First, Python type analysis requires models that explicitly embrace dynamism rather than attempting to restrict or approximate it away. Second, the integration of runtime evidence—through tracing, selective instrumentation, or lightweight profiling—appears essential for achieving practical precision. Third, AI-based reasoning should not be treated merely as a replacement for static methods; instead, it may serve as a complementary inferential layer capable of synthesizing type information beyond the reach of rule-based systems. Finally, the long-term direction for Python type analysis likely lies in hybrid, cyclic, or feedback-driven architectures that iteratively refine static approximations with empirical runtime knowledge and AI-assisted inference.

Overall, the central challenge in Python type analysis remains the discontinuity between the language’s dynamically evolving behavior and the goals of static safety and predictability. While no single method provides a complete solution, combining static reasoning, runtime observation, and AI-based inference creates a more holistic framework capable of addressing Python’s unique characteristics. This observation underscores a growing consensus in recent work:

Python type analysis is not a problem solvable by any one paradigm, but rather a field that demands integrated, multi-modal approaches.

V. CONCLUSION

This survey systematically reviewed the technical trends and research directions in Python type analysis. Python’s dynamic execution model fundamentally conflicts with the fixed and closed type structures assumed by traditional static analyses, making it challenging to build practical analyzers by simply adapting type analysis techniques from other languages. Consequently, the Python ecosystem has seen the parallel development of a wide range of strategies, including static analyzers, runtime monitoring systems, execution trace-based type inference, symbolic execution, constraint solving, and more recently, LLM-based approaches.

From this analysis, several conclusions can be drawn:

- Static analysis alone cannot fully capture Python’s dynamic features. Structural typing, dynamic attributes, reflection, and heterogeneous containers violate assumptions of conventional static type theory.
- Dynamic and hybrid approaches provide high accuracy but cannot fully resolve coverage and performance limitations.
- LLM-based approaches offer new possibilities but still fail to guarantee soundness.

Therefore, a hybrid, multi-layered approach that combines the scalability of static analysis, the precision of dynamic analysis, and the generalization capability of AI-based inference is essential in Python. Overall, Python type analysis remains an evolving domain, situated in the tension between the language’s dynamic design philosophy and the requirements of static safety, with each technique playing a complementary role in bridging this gap. Future research is expected to formalize the interaction between static, dynamic, and AI-based techniques, exploring new analysis models that balance soundness, precision, and scalability.

REFERENCES

- [1] Tiobe index: The python programming language, <https://www.tiobe.com/tiobe-index/python/>.
- [2] Tensorflow: An end-to-end platform for machine learning, <https://www.tensorflow.org/>.
- [3] Pytorch, <https://pytorch.org/>.
- [4] Hugging face-transformers, <https://huggingface.co/docs/transformers/index>.
- [5] W. Oh, H. Oh, Towards effective static type-error detection for python, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE ’24, Association for Computing Machinery, New York, NY, USA, 2024, p. 1808–1820. doi:10.1145/3691620.3695545. URL <https://doi.org/10.1145/3691620.3695545>
- [6] F. K. et al., An empirical study of type-related defects in python projects, IEEE Transactions on Software Engineering (2021).
- [7] Pep 484 - type hints, <https://peps.python.org/pep-0484/>.
- [8] Mypy: Static typing for python, <https://github.com/python/mypy>.
- [9] pyre-check: Performant type-checking for python, <https://github.com/facebook/pyre-check>.
- [10] pyright: Static type checker for python, <https://github.com/microsoft/pyright>.
- [11] J. Siek, W. Taha, Gradual typing for functional languages, 2006.
- [12] T. Dong, L. Chen, Z. Xu, B. Yu, Static type analysis for python, in: 2014 11th Web Information System and Application Conference, 2014, pp. 65–68. doi:10.1109/WISA.2014.20.
- [13] M. Hassan, C. Urban, M. Eilers, P. Müller, Maxsmt-based type inference for python 3, in: H. Chockler, G. Weissenbacher (Eds.), Computer Aided Verification, Springer International Publishing, Cham, 2018, pp. 12–19.
- [14] S. Cui, G. Zhao, Z. Dai, L. Wang, R. Huang, J. Huang, Pyinfer: Deep learning semantic type inference for python variables, CoRR abs/2106.14316 (2021). arXiv:2106.14316. URL <https://arxiv.org/abs/2106.14316>
- [15] J. Wei, M. Goyal, G. Durrett, I. Dillig, Lambdanet: Probabilistic type inference using graph neural networks, CoRR abs/2005.02161 (2020). arXiv:2005.02161. URL <https://arxiv.org/abs/2005.02161>
- [16] J. Wei, G. Durrett, I. Dillig, Typet5: Seq2seq type inference using static analysis, in: The Eleventh International Conference on Learning Representations, 2023. URL <https://openreview.net/forum?id=4TyNEhI2GdN>
- [17] Y. Peng, C. Gao, Z. Li, B. Gao, D. Lo, Q. Zhang, M. Lyu, Static inference meets deep learning: a hybrid type inference approach for python, in: Proceedings of the 44th International Conference on Software Engineering, ICSE ’22, Association for Computing Machinery, New York, NY, USA, 2022, p. 2019–2030. doi:10.1145/3510003.3510038. URL <https://doi.org/10.1145/3510003.3510038>
- [18] K. Sun, Y. Zhao, D. Hao, L. Zhang, Static type recommendation for python, in: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, ASE ’22, Association for Computing Machinery, New York, NY, USA, 2023. doi:10.1145/3551349.3561150. URL <https://doi.org/10.1145/3551349.3561150>
- [19] Y. Yan, Y. Feng, H. Fan, B. Xu, Dlinfer: Deep learning with static slicing for python type inference, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), 2023, pp. 2009–2021. doi:10.1109/ICSE48619.2023.00170.
- [20] W. Oh, H. Oh, Pyter: effective program repair for python type errors, in: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Association for Computing Machinery, New York, NY, USA, 2022, p. 922–934. doi:10.1145/3540250.3549130. URL <https://doi.org/10.1145/3540250.3549130>
- [21] Y. W. Chow, L. Di Grazia, M. Pradel, Ptyt: Repairing static type errors in python, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE ’24, Association for Computing Machinery, New York, NY, USA, 2024. doi:10.1145/3597503.3639184. URL <https://doi.org/10.1145/3597503.3639184>
- [22] J. Wu, C. Lemieux, Quac: Quick attribute-centric type inference for python, Proc. ACM Program. Lang. 8 (OOPSLA2) (Oct. 2024). doi:10.1145/3689783. URL <https://doi.org/10.1145/3689783>
- [23] typeguard: Run-time type checker for python, <https://github.com/agronholm/typeguard>.
- [24] Monkeytype: A python library that generates static type annotations by collecting runtime types, <https://github.com/Instagram/MonkeyType>.
- [25] Crosshair: An analysis tool for python that blurs the line between testing and type systems., <https://github.com/pschanely/CrossHair>.
- [26] F. Ortin, J. B. G. Perez-Schofield, J. M. Redondo, Towards a static type checker for python, in: European Conference on Object-Oriented Programming (ECOOP), Scripts to Programs Workshop, STOP, Vol. 15, 2015, pp. 1–2.
- [27] Z. Xu, X. Zhang, L. Chen, K. Pei, B. Xu, Python probabilistic type inference with natural language support, in: Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE 2016, Association for Computing Machinery, New York, NY, USA, 2016, p. 607–618. doi:10.1145/2950290.2950343. URL <https://doi.org/10.1145/2950290.2950343>
- [28] V. J. Hellendoorn, C. Bird, E. T. Barr, M. Allamanis, Deep learning type inference, in: Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2018, Association for Computing Machinery, New York, NY, USA, 2018, p. 152–162. doi:

10.1145/3236024.3236051.

URL <https://doi.org/10.1145/3236024.3236051>

- [29] M. Pradel, G. Gousios, J. Liu, S. Chandra, Typewriter: neural type prediction with search-based validation, in: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2020, Association for Computing Machinery, New York, NY, USA, 2020, p. 209–220. doi:10.1145/3368089.3409715. URL <https://doi.org/10.1145/3368089.3409715>
- [30] M. Hu, Y. Zhang, W. Huang, Y. Xiong, Static type inference for foreign functions of python, in: 2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE), 2021, pp. 423–433. doi:10.1109/ISSRE52982.2021.00051.
- [31] A. M. Mir, E. Latoškinas, S. Proksch, G. Gousios, Type4py: practical deep similarity learning-based type inference for python, in: Proceedings of the 44th International Conference on Software Engineering, ICSE '22, Association for Computing Machinery, New York, NY, USA, 2022, p. 2241–2252. doi:10.1145/3510003.3510124. URL <https://doi.org/10.1145/3510003.3510124>
- [32] J. Elkobi, B. Gruner, T. Sonnekalb, C.-A. Brust, Typical - type inference for python in critical accuracy level, in: 2023 IEEE/ACIS 21st International Conference on Software Engineering Research, Management and Applications (SERA), 2023, pp. 16–21. doi:10.1109/SERA57763.2023.10197800.
- [33] R. Monat, A. Ouadjaout, A. Miné, Static Type Analysis by Abstract Interpretation of Python Programs, in: R. Hirschfeld, T. Pape (Eds.), 34th European Conference on Object-Oriented Programming (ECOOP 2020), Vol. 166 of Leibniz International Proceedings in Informatics (LIPIcs), Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 2020, pp. 17:1–17:29. doi:10.4230/LIPIcs.ECOOP.2020.17. URL <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2020.17>
- [34] pytype: A static type analyzer for python code, <https://github.com/google/pytype>.